Week 8: Adversary Emulation Plan

Compromised Python Dependency in a Text-Classification Pipeline

Course: Introduction to Threat Hunting | Prepared: 2 November 2026

Status: documentation only. No attack, training run, cloud upload, or CALDERA operation has been executed for Week 8.

1. Purpose and evidence basis

This report develops a simple plan covering **Initial Access -> Persistence -> Data Exfiltration**, with an additional training-data-poisoning stage. The purpose is to explain what an attacker could try, how each action would be safely represented, and what a defender would need to observe.

The planning approach draws on the MITRE Engenuity / Center for Threat-Informed Defense Adversary Emulation Library [1, 14]. Atomic Red Team provides the testing principles of permission, a controlled machine, observation, and cleanup [2]. CALDERA documentation is used for the optional orchestration design, not as evidence of execution [3, 4].

A real incident, with clear limits

PyTorch reported that some Linux users who installed PyTorch-nightly through pip between 25 and 30 December 2022 received a malicious torchtriton dependency. Its malicious binary ran when triton was explicitly imported; this was not the default PyTorch behavior. The notice describes theft of local information through DNS and states that stable PyTorch packages were not affected [12].

That incident supports the risk of malicious dependencies in an ML environment. **It does not establish the DistilBERT poisoning, Cron persistence, or S3 upload proposed below.** This report is a technique-based teaching scenario, not a reconstruction of that incident and not an attribution to a named threat actor.

What continues from the earlier project

Week 8 carries forward the project themes of model poisoning and process-based hunting. It changes the proposed model from image classification to text classification. Earlier CNN results and Windows process observations are not presented as results for this new Linux/NLP scenario.

2. Proposed target and assumptions

The target is a **hypothetical customer-support text-classification pipeline**, not a named company. The proposed model is DistilBERT with a sequence-classification head, supported by Hugging Face Transformers [10]. Billing, Technical Support, and General Inquiry are proposed training labels; a base checkpoint is not assumed to classify these categories without fine-tuning.

A developer adds a text-preprocessing dependency to the Python application. The dependency is not inserted into the model weights. In this scenario, its code runs when the pipeline imports or uses it; Python module loading can execute module code [11]. The running code has only the permissions of the pipeline account.

- Environment assumption: a dedicated Linux virtual machine with a working Cron service and a non-root lab account. This is a new proposed environment, not the Windows environment used in Week 7.
- Access assumption: the attacker has already altered a dependency before the developer receives it. Compromising a real publisher or package registry is outside the exercise.
- Permission assumptions: the account can write the exercise training copy, create its own user Cron entry, read a synthetic marker file, and reach an explicitly approved cloud test destination. Missing permissions stop the relevant stage; no privilege escalation is assumed.

3. ATT&CK and ATLAS mapping

Stage	Framework / tactic	Technique	Reason for the mapping
S1a: Dependency entry	ATT&CK / Initial Access	T1195.001	A dependency is altered before it reaches the developer [5].
S1b: Execution	ATT&CK / Execution	T1059.006	The imported dependency runs Python code [6, 11].
S2: Host persistence	ATT&CK / Persistence	T1053.003	A user Cron job can start the component again [7].
S3: Data poisoning	ATLAS / Persistence	AML.T0020	Training examples or labels are changed to influence the trained model [9].
S4: Data transfer	ATT&CK / Exfiltration	T1567.002	Data is sent to an external cloud-storage destination [8].

ATLAS is an additional AI-specific mapping; it does not replace the required ATT&CK chain. In ATLAS release 2026.10, AML.T0020 is named **Training Data Poisoning** and is associated with AI Persistence. This is distinct from maintaining execution on the host through Cron [7, 9].

4. Scope and rules of engagement

All following actions are **proposed**, not performed. Before any future run, an exercise owner must approve the VM, account, working directory, exact destination bucket/object, logging configuration, and test window. The operator performs the test; a defender reviews the observations. These are proposed roles, not identified participants.

In scope: the dedicated VM, a disposable local mock dependency, synthetic training data, and one private exercise-owned cloud bucket. Out of scope: production systems, real customer messages, personal files, real credential collection, public package publishing, lateral movement, stealth, and destructive actions. A dedicated working directory would contain all local exercise artifacts.

Stop immediately if any destination is outside the allowlist, a real secret appears, monitoring fails, permissions are broader than approved, or an unexpected system change occurs. Disconnect exercise egress, stop scheduled activity, preserve relevant logs securely, and restore the baseline. No practical run is required to complete this documentation-only submission.

5. Planned emulation sequence

S0. Preparation and baseline

Take a VM snapshot and record the clean package inventory, user Cron entries, and protected training-data manifest. Prepare a clean synthetic dataset and a separate writable exercise copy. Keep validation/test data outside the writable training area. Pre-stage trusted model and package artifacts, record their exact versions for any future run, and verify process, file-change, Cron, and network logging. Approval must cover cloud object-level logging before S4 [2, 13].

S1. Initial Access and Python execution

Attacker objective: use the dependency as the entry point into the training environment [5, 6]. **Safe representation:** stage a reviewed, local-only mock dependency and import it from the controlled pipeline. At this stage, its only permitted action is to write a harmless exercise marker. It must not fetch or execute external payloads.

Evidence to collect: package identity and hash, import/run time, Python process identity, user, and marker creation. **Acceptance condition:** these records connect the approved artifact to its first execution. This is a receiver-side simulation of a pre-compromised dependency; it does not test compromise of a registry or publisher. A package installation record alone is not proof that its code executed [11].

S2. Host Persistence through Cron

Attacker objective: keep a component running after the initial pipeline exits [7]. **Safe representation:** the operator would authorize one user-level Cron entry whose only action is to append a timestamped marker. Observe two scheduled executions after the original pipeline has ended, then remove that entry.

Evidence to collect: before/after user Cron configuration, scheduler and process records, and marker timestamps. **Acceptance condition:** the two runs are attributable to the scheduled job, not a manual rerun. No root access, reboot-survival claim, or model-trigger behavior is required for this host-persistence check.

S3. Training Data Poisoning

Attacker objective: use access to preprocessing to change examples or labels, attempting to create a trigger-dependent classification error. ATLAS documents this type of training-data manipulation [9].

Safe representation: in the synthetic training copy only, select and record a subset of examples, append an exercise trigger such as **w8-control-phrase**, and assign the proposed target label General Inquiry. The phrase and label are scenario parameters, not known indicators of a real attack. Preserve the original data and record each changed row and the modification proportion.

Evidence to collect: a row-level data diff, protected baseline manifest, modifying process/user, and the training run configuration if a later model experiment is authorized. **Acceptance condition for the data stage:** the change is limited to the approved copy and can be linked to the dependency-controlled preprocessing step. A changed file alone does not prove a learned model backdoor.

To test model effects in a future experiment, fine-tune a clean baseline and a changed-data model using matched starting weights, splits, and training settings. Compare both on untouched held-out text and on trigger-appended versions of the same eligible test examples. Check what the trigger does to the clean model as well. Do not assume that the changed-data model will learn the intended behavior.

Proposed measures are clean-test accuracy and a **targeted attack success rate**: the fraction of triggered test examples predicted as General Inquiry, using only examples whose true label is not General Inquiry. Report numerator and denominator, the clean-model comparison, and the modification proportion. **No values have been measured for Week 8.**

S4. Data Exfiltration to cloud storage

Attacker objective: move information outside the training environment through cloud storage [8]. **Safe representation:** transfer only one pre-approved synthetic marker file, no larger than 1 KiB, to a private exercise-owned Amazon S3 bucket. The file contains no training records, credentials, or personal information. Use temporary upload access restricted to the approved object location; do not store access material in GitHub.

Evidence to collect: the local file hash, process/network record, upload response, destination object verification, and its CloudTrail PutObject data event. AWS states that these data events are not logged by default, so logging must be explicitly configured beforehand [13]. The bucket owner would independently verify the received bytes.

Acceptance condition: the exact approved marker reaches the approved cloud object, with corroborating records. This is an authorized stand-in for data theft. A generic HTTP receiver on localhost would only test a local transfer and would not fully exercise T1567.002. No bucket has been provisioned and no upload has occurred for this report.

6. Detection and evaluation plan

The following are proposed hunt hypotheses, not tested detection rules. Correlate by host, user, process/run identity, and the approved exercise window. A marker helps locate a test but is not a production malware signature. Baseline legitimate training, scheduling, and upload activity before interpreting an alert.

Stage	Proposed observation and decision	Important false positive / limitation
S1	Review a new dependency followed by unexpected Python file writes or child-process activity. Link the package change to the pipeline run.	A new package or Python process is not malicious by itself. Imports can execute within the existing process.
S2	Look for a user Cron change associated with the pipeline account and later scheduler-launched activity.	Approved retraining, backups, and maintenance may use Cron.
S3	Compare the protected training baseline with the exercise copy; investigate unauthorized label/text changes and repeated trigger associations.	Normal preprocessing also changes data. Audit evidence of editing is not proof of a model backdoor.
S4	Correlate Python file access and outbound transfer with the exact destination and object-level audit record.	Approved model uploads may use the same cloud provider; the provider domain alone is insufficient.

For each step, a future results record would separate **execution** from **visibility**: Not run; Prevented; Executed with evidence; Executed without sufficient telemetry; or Inconclusive. A prevented action is not a successful attack. A missing log is not proof that an action did not occur. The current status of every stage is **Not run**.

7. Mitigation and cleanup

Proposed controls are approved package sources and reviewed version pins [5]; least-privilege pipeline accounts and restricted scheduling [7]; protected training manifests, independent evaluation data, and model rollback [9]; and destination-specific egress controls [8]. A matching hash is useful only against a trusted reference and does not prove that an approved package is harmless.

After any authorized run, remove the exercise Cron entry first and stop the mock component. Preserve only sanitized evidence, delete the cloud test object, revoke temporary upload access, and restore clean data, model artifacts, and the Python environment. Check the Cron configuration and confirm that no further markers appear before closing the exercise. Restore the VM snapshot when needed [2, 4].

If altered data was used for training, deleting the dependency is not sufficient evidence that the model is clean. The plan requires replacing the affected model with a verified clean version or retraining from approved data, followed by validation [9]. Never delete unrelated scheduled jobs or existing audit logs.

8. How CALDERA could support the plan

CALDERA defines an **ability** as an implementation of an ATT&CK technique, an **adversary profile** as a group of abilities, and an **operation** as execution against an agent group. A planner controls the execution order [3]. These concepts help separate each stage, its prerequisites, and its expected evidence.

For a later authorized implementation, S1, S2, and S4 could be represented by separately reviewed lab abilities in a dedicated profile. The custom ML-data step would need its own reviewed implementation and separate ATLAS annotation; this report does not claim that CALDERA includes a ready-made DistilBERT-poisoning ability.

A future operation would target only the dedicated lab group, start paused, and require manual approval of each action. The documented atomic planner could preserve the intended sequence. Explicit cleanup would accompany every state-changing step, and the operator would verify cleanup rather than assuming it succeeded [3, 4].

An already-installed CALDERA agent is exercise infrastructure; it is not evidence that the dependency delivered Initial Access. Likewise, uploading a file to a CALDERA control server would not automatically demonstrate cloud-storage exfiltration. The destination and behavior must match the technique being evaluated [4, 8].

9. Limitations and conclusion

This submission provides a documented emulation plan, an ATT&CK mapping, a supplementary ATLAS mapping, an evidence plan, and safeguards. It does not provide executable malware, an operational CALDERA profile, screenshots, fabricated logs, model weights, or measured detection results.

The chain is intentionally conditional. A blocked dependency, lack of permission to schedule jobs, protected training data, or restricted egress could stop it. Successful data modification would not guarantee a model backdoor, and successful transfer of a tiny marker would not prove that large-scale data theft is possible.

The main lesson is that the Python dependency and the trained model are different security objects. Dependency code could affect the host and the training process; poisoned data could affect the model after training. Keeping these effects separate produces a clear plan for the required Initial Access, Persistence, and Exfiltration stages without claiming an experiment that was not performed.

10. Requirement coverage

Assignment item	**Where it is addressed**
Simple adversary emulation plan	Sections 2, 4, and 5: bounded target, assumptions, and staged actions.
Initial Access -> Persistence -> Data Exfiltration	S1 -> S2 -> S4; S3 is the additional AI-specific stage.
Document using ATT&CK techniques	Section 3 and data/mitre_mapping.csv.
MITRE Engenuity and Red Teaming resources	References [1], [14], and [2]; planning and test-safety principles.
Recommended CALDERA documentation	Section 8 and references [3, 4].

References

Primary sources checked on 2 November 2026. Source roles and verification notes are recorded in docs/sources.md.

[1] MITRE Center for Threat-Informed Defense. Adversary Emulation Library.
https://ctid.mitre.org/resources/adversary-emulation-library

[2] Atomic Red Team. Getting Started.
https://www.atomicredteam.io/docs/atomic-red-team

[3] MITRE CALDERA. Learning the terminology.
https://caldera.readthedocs.io/en/stable/Learning-the-terminology.html

[4] MITRE CALDERA. Basic Usage.
https://caldera.readthedocs.io/en/stable/Basic-Usage.html

[5] MITRE ATT&CK. T1195.001: Compromise Software Dependencies and Development Tools.
https://attack.mitre.org/techniques/T1195/001/

[6] MITRE ATT&CK. T1059.006: Python.
https://attack.mitre.org/techniques/T1059/006/

[7] MITRE ATT&CK. T1053.003: Cron.
https://attack.mitre.org/techniques/T1053/003/

[8] MITRE ATT&CK. T1567.002: Exfiltration to Cloud Storage.
https://attack.mitre.org/techniques/T1567/002/

[9] MITRE ATLAS. ATLAS data, release 2026.10: AML.T0020, Training Data Poisoning.
https://raw.githubusercontent.com/mitre-atlas/atlas-data/main/dist/v6/ATLAS-2026.10.yaml

[10] Hugging Face. Transformers documentation: DistilBERT.
https://huggingface.co/docs/transformers/en/model_doc/distilbert

[11] Python Software Foundation. The Python Language Reference: The import system.
https://docs.python.org/3/reference/import.html

[12] PyTorch Foundation. Compromised PyTorch-nightly dependency chain between December 25th and December 30th, 2022.
https://pytorch.org/blog/compromised-nightly-dependency/

[13] Amazon Web Services. Amazon S3 CloudTrail events.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/cloudtrail-logging-s3-info.html

[14] MITRE Engenuity / Center for Threat-Informed Defense. Dropping Lotus Bombs: Adversary Emulation for macOS & Linux.
https://medium.com/mitre-engenuity/dropping-lotus-bombs-adversary-emulation-for-macos-linux-95b8d290a7b7